🧠 COS30008 – Practice 2

Time: 2 hours
Total Marks: 396
Instructions: Implement all required methods according to specifications. Use provided test drivers to verify your implementation.

🔷 Problem 1: Simplified Binary Search Tree Infrastructure (120 marks)

Implement a basic BST with only essential operations and simplified logic.

Required methods:

- `bool insert(const T& aKey);`
- `bool remove(const T& aKey);`
- `const T* find(const T& aKey) const;`
- `BinarySearchTree(const T& aKey);`
- `~BinarySearchTree();`
- `const T& operator*() const;`
- `bool empty() const;`
- `bool leaf() const;`
- `size_t size() const;`
- `size_t depth() const;`

Specifications:

- No need to handle duplicate keys.
- `remove()` only needs to handle leaf and single-child nodes.
- `depth()` and `size()` can be implemented recursively without optimization.
- `operator*()` throws `domain_error` for NIL trees.

🔷 Problem 2: Basic Copy Control (64 marks)

Implement basic copy control without deep metadata or exception handling.

Required methods:

- `BinarySearchTree(const BST& aOtherBST);`
- `BST& operator=(const BST& aOtherBST);`
- `BST* clone() const;`

Specifications:

- Copy constructor and assignment operator should perform deep copy.
- NIL trees can be copied without throwing exceptions.
- `clone()` returns a new tree with the same structure.

🔷 Problem 3: Move Semantics (72 marks)

Implement move semantics without logging or advanced resource tracking.

Required methods:

- `BinarySearchTree(T&& aKey);`
- `BinarySearchTree(BST&& aOtherBST);`
- `BST& operator=(BST&& aOtherBST);`

Specifications:

- Move constructor and assignment transfer ownership.
- Source tree becomes empty after move.
- No need to log or track moved keys.

🔷 Problem 4: In-order Iterator (90 marks)

Implement a basic in-order iterator for BST traversal.

Required iterator class structure:

```
template<typename T>
class BinarySearchTreeInOrderIterator {
private:
    using BST = BinarySearchTree<T>;
    using BSTNode = BST*;
    using NodeStack = std::stack<const BSTNode>;
```

```
    const BST* fBST;
    NodeStack fStack;
    void pushLeftPath(const BST* aNode);
public:
    BinarySearchTreeInOrderIterator(const BST* aBST);
    const T& operator*() const;
    BinarySearchTreeInOrderIterator& operator++();
    BinarySearchTreeInOrderIterator operator++(int);
    bool operator==(const BinarySearchTreeInOrderIterator& aOther) const;
    bool operator!=(const BinarySearchTreeInOrderIterator& aOther) const;
};
```

Specifications:

- Traverse nodes in ascending order.
- Add `begin()` and `end()` methods to BST.
- No need for range-based loop support.

🔷 Problem 5: Theoretical Questions (50 marks)

Answer the following in 1–2 sentences.

a. **What is a binary search tree and why is it useful?** [4]
b. **What is the difference between copying and cloning an object in C++?** [6]
c. **Why should we use destructors in C++ classes?** [4]
d. **What is a template in C++ and how does it help with generic programming?** [6]
e. **What does the `const` keyword mean in member functions?** [4]
f. **What is the purpose of the `nullptr` keyword in C++?** [4]
g. **What is a move constructor and when is it used?** [6]
h. **What is the time complexity of searching in a BST?** [4]
i. **Why is recursion useful in tree-based algorithms?** [4]
j. **How can we find the minimum value in a BST?** [8]